

Cryptography and Network Security

Integrated Situation — Technical Report

ETTCS801 | ULK Polytechnic Institute

[Your full name]
[Registration number]

September 24, 2026

Contents

1	Introduction	1
2	Risk Assessment	1
2.1	Assets, vulnerabilities and consequences	1
2.2	Risk ranking	2
2.3	Recommended controls	2
3	Encryption and Integrity Application	2
3.1	Design	2
3.2	Test evidence	3
4	Network Traffic Filtering	4
4.1	Laboratory setup	4
4.2	Firewall rules	4
4.3	Test procedure and results	4
5	Conclusion	4

1 Introduction

A polytechnic institute stores student records on a central server and transfers files between two campuses. A security review found weak staff passwords, outdated software, unencrypted file transfers and guest-network access to the records server, and repeated connection attempts from an unfamiliar external address were observed. This report documents the risk assessment, the Python encryption and integrity toolkit, and the firewall rules produced to address these findings. All source files are in the GitHub repository (Section 5).

2 Risk Assessment

2.1 Assets, vulnerabilities and consequences

#	Asset	Vulnerability	Possible consequence
R1	Staff accounts and credentials	Weak passwords; repeated external login attempts	Account takeover; records read, altered or deleted; misuse under a staff identity.
R2	Student records server	Guest network can reach the server; outdated software	Scanning and exploitation of known flaws; data breach, ransomware, outage; reputational and legal harm.
R3	Record files in transit between campuses	Unencrypted file transfer	Eavesdropping or man-in-the-middle; records disclosed or silently modified.

2.2 Risk ranking

Likelihood and impact are scored from 1 (low) to 5 (high); the score is their product.

Rank	Risk	Likelihood	Impact	Score
1	R1 – Weak staff passwords	5	5	25
2	R2 – Guest access / outdated software	4	5	20
3	R3 – Unencrypted transfers	3	4	12

Reasoning. R1 ranks first because attacks are already occurring and weak passwords need little skill to defeat; one compromised account exposes all data the user can reach. R2 is second: guests are many and untrusted, and known exploits exist for outdated software, with very high impact because all records reside on one server. R3 ranks third because interception requires a position on the inter-campus path, although success exposes and allows tampering with complete files.

2.3 Recommended controls

Risk	Control
R1	Strong password policy (minimum 12 characters, no reuse), multi-factor authentication, account lockout / rate limiting.
R2	Network segmentation with firewall rules denying guest access to the server, plus a regular patching schedule.
R3	Encrypt files before transfer (AES-256-GCM) and use secure channels (SFTP/TLS/VPN), with SHA-256 integrity checks.

3 Encryption and Integrity Application

3.1 Design

The programme `src/secure_records.py` is written in Python and uses the `cryptography` library.

- **Encryption** – AES-256-GCM authenticated encryption. A random 256-bit key is generated once and stored *outside* the repository (default `~/.ulk_keys/records.key`); the tool refuses a key path inside a git repository. Each encryption uses a fresh random 96-bit nonce. The output file is `header (4 bytes) || nonce (12 bytes) || ciphertext || GCM tag (16 bytes)`.
- **Decryption** – reads the nonce, decrypts and authenticates. A wrong key or any modified byte raises an authentication failure, reported as a clean error message. With `-original` the decrypted file's SHA-256 is compared with the original's to prove the contents match.

- **Integrity** – hash stores the SHA-256 digest of a file in a baseline file (`hashes.json`); check recomputes the digest and reports *UNCHANGED* or *MODIFIED*.
- **Error handling** – missing files, directories, empty files, invalid or missing keys, corrupt or non-encrypted input and corrupt baselines are caught and reported with a message and exit code, never a traceback.

3.2 Test evidence

Ten automated unit tests (round trip, ciphertext differs from plaintext, tamper detection, wrong key, missing key, missing/empty/invalid input, hash change detection, CLI error code) all pass:

```
$ python3 -m unittest discover -s tests -v
...
Ran 10 tests in 0.008s
OK
```

Full demonstration output (`tests/run_demo.sh`); the two SHA-256 values match after decryption, and the modified file is flagged:

```
=== 1. Generate key (stored outside repo) ===
=== 2. Encrypt sample student record file ===

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key encrypt data/sample_students.csv /tmp/tmp.zuU1TeF3GB/students.enc
[OK] Encrypted data/sample_students.csv -> /tmp/tmp.zuU1TeF3GB/students.enc
    Plaintext SHA-256: df749f57bf7f8e53593ebe7f7f977998f9da196c671e884aaa15c9dc7a526423
(exit code: 0)

$ od -An -tx1 -N32 /tmp/tmp.zuU1TeF3GB/students.enc
55 4c 4b 31 dd ed 3d 44 20 67 cc 89 7f 8a 57 87
22 41 80 b3 e4 aa a2 9a 1e ac f3 00 c2 7c 82 02
(exit code: 0)
=== 3. Decrypt and verify against original ===

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key decrypt /tmp/tmp.zuU1TeF3GB/students.enc /tmp/tmp.zuU1TeF3GB/
students_decrypted.csv --original data/sample_students.csv
[OK] Decrypted /tmp/tmp.zuU1TeF3GB/students.enc -> /tmp/tmp.zuU1TeF3GB/students_decrypted.csv
    Plaintext SHA-256: df749f57bf7f8e53593ebe7f7f977998f9da196c671e884aaa15c9dc7a526423
    Original SHA-256: df749f57bf7f8e53593ebe7f7f977998f9da196c671e884aaa15c9dc7a526423
[OK] MATCH: decrypted file is identical to the original.
(exit code: 0)
=== 4. SHA-256 baseline and change detection ===

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key hash /tmp/tmp.zuU1TeF3GB/students_live.csv --baseline /tmp/tmp.
zuU1TeF3GB/hashes.json
[OK] SHA-256(/tmp/tmp.zuU1TeF3GB/students_live.csv) = df749f57bf7f8e53593ebe7f7f977998f9da196c671e884aaa15c9dc7a526423
    Stored in /tmp/tmp.zuU1TeF3GB/hashes.json
(exit code: 0)

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key check /tmp/tmp.zuU1TeF3GB/students_live.csv --baseline /tmp/tmp.
zuU1TeF3GB/hashes.json
    Expected: df749f57bf7f8e53593ebe7f7f977998f9da196c671e884aaa15c9dc7a526423
    Current : df749f57bf7f8e53593ebe7f7f977998f9da196c671e884aaa15c9dc7a526423
[OK] UNCHANGED: file integrity verified.
(exit code: 0)
(file modified: one record appended)

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key check /tmp/tmp.zuU1TeF3GB/students_live.csv --baseline /tmp/tmp.
zuU1TeF3GB/hashes.json
    Expected: df749f57bf7f8e53593ebe7f7f977998f9da196c671e884aaa15c9dc7a526423
    Current : 8dfa0edb9b29f3c01ea70236be8c44f678e850336d5e967f9887df6ff75e204
[ALERT] MODIFIED: file has changed since the baseline was recorded!
(exit code: 2)
=== 5. Error handling ===

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key encrypt does_not_exist.csv /tmp/tmp.zuU1TeF3GB/x.enc
[ERROR] File not found: does_not_exist.csv
(exit code: 1)

$ python3 src/secure_records.py --key-path /nonexistent.key decrypt /tmp/tmp.zuU1TeF3GB/students.enc /tmp/tmp.zuU1TeF3GB/x.csv
[ERROR] File not found: /nonexistent.key
(exit code: 1)

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key decrypt /tmp/tmp.zuU1TeF3GB/bad.enc /tmp/tmp.zuU1TeF3GB/x.csv
[ERROR] /tmp/tmp.zuU1TeF3GB/bad.enc is not a valid encrypted file (wrong format or truncated).
(exit code: 1)

$ python3 src/secure_records.py --key-path /tmp/demo_keys/records.key encrypt /tmp/tmp.zuU1TeF3GB/empty.csv /tmp/tmp.zuU1TeF3GB/x.enc
[ERROR] Input file is empty: /tmp/tmp.zuU1TeF3GB/empty.csv
(exit code: 1)
```

4 Network Traffic Filtering

4.1 Laboratory setup

[Describe your lab: server IP, guest network, staff network, other/external client, firewall tool (iptables), and the service the assessor specified.]

4.2 Firewall rules

Rules are in `firewall/records_server_firewall.sh` and use a dedicated iptables chain so they can be removed cleanly. In order of evaluation:

1. Accept established/related traffic (and loopback) so existing sessions keep working.
2. **Block guest → server (all ports):** `-s GUEST_NET -j DROP` (logged, rate limited).
3. **Permit staff → named service:** `-s STAFF_NET -p tcp -dport PORT -j ACCEPT`.
4. **Block all other inbound access to the service:** `-p tcp -dport PORT -j DROP`.

```
iptables -A RECORDS_FILTER -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A RECORDS_FILTER -s $GUEST_NET -j DROP
iptables -A RECORDS_FILTER -s $STAFF_NET -p tcp --dport $SERVICE_PORT \
-m conntrack --ctstate NEW -j ACCEPT
iptables -A RECORDS_FILTER -p tcp --dport $SERVICE_PORT -j DROP
iptables -A RECORDS_FILTER -j RETURN
iptables -I INPUT 1 -j RECORDS_FILTER
```

[Replace variables with your real values if you want the report to show them.]

4.3 Test procedure and results

One permitted and two blocked connections were tested with `nc -zv -w 5 SERVER_IP PORT`.

Test	Source	Expected	Actual
T1 permitted	Staff client	Connection succeeds	[paste result]
T2 blocked	Guest client	Timeout / dropped	[paste result]
T3 blocked	Other client	Timeout / dropped	[paste result]

[Paste the terminal output or a screenshot from `filter_tests.md`, and the iptables packet counters.]

5 Conclusion

The assessment ranked weak passwords, guest access to the records server, and unencrypted transfers as the main risks. The toolkit addresses the last two directly: authenticated AES-256-GCM encryption protects files in transit and at rest, SHA-256 baselines detect tampering, and the firewall removes the guest path to the server while limiting the service to the staff network. Weak passwords are addressed by policy and MFA, which are organisational controls outside the code. Limitations: the key must be distributed securely between campuses (a key-management system would be needed at scale), and firewall rules cover only the tested service. Future work: automated patching, centralised logging and intrusion detection.

GitHub repository: <https://github.com/YOUR-USERNAME/cryptography-network-security-exam>